

Proje 4: RBFS ve SMA*

2021221054 - MUHAMMED EMİN KORKUNÇ

Problem Tanımı: Bir harita üzerinde şehirler arası yolların maliyetlerini ve bağlantılarını içeren bir grafik var. Amacımız başlangıç şehrinde hedef şehre en kısa yolu bulmak. Yolların maliyeti, her iki şehir arasındaki mesafeyi temsil ediyor.

Şehirler: A, B, C, D, E, F

Bağlantılar ve maliyetler:

A → B: 1

A → C: 4

B → D: 2

B → E: 5

C → F: 3

D → E: 1

E → F: 1

```
import heapq
import math

class Graph:
    def __init__(self):
        self.graph = {}

    def add_edge(self, u, v, cost):
        if u not in self.graph:
            self.graph[u] = []
        self.graph[u].append((v, cost))

    def neighbors(self, node):
        return self.graph.get(node, [])

def rbfs(graph, start, goal, f_limit, h):
    def recursive_best_first(node, g, f_limit):
        if node == goal:
            return [node], g

        successors = []
```

```

    for neighbor, cost in graph.neighbors(node):
        f_value = g + cost + h(neighbor)
        successors.append((f_value, neighbor, g + cost))
    if not successors:
        return None, math.inf

    successors.sort() # Sort by f-value
    while successors:
        best_f, best_node, best_g = successors[0]
        if best_f > f_limit:
            return None, best_f

        if len(successors) > 1:
            alternative = successors[1][0]
        else:
            alternative = math.inf

        result, best_f = recursive_best_first(best_node, best_g, min(f_limit,
alternative))
        successors[0] = (best_f, best_node, best_g)
        successors.sort()
        if result:
            return [node] + result, best_f

    return None, math.inf

return recursive_best_first(start, 0, f_limit)

def sma_star(graph, start, goal, h):
    open_list = []
    closed = {}
    heapq.heappush(open_list, (h(start), start, 0, None))
    max_open_size = 5

    while open_list:
        if len(open_list) > max_open_size:
            open_list.pop()

        f, node, g, parent = heapq.heappop(open_list)

        if node == goal:
            path = []
            while parent:
                path.append(node)
                node, parent = parent

```

```

        path.append(start)
        return path[::-1]

    closed[node] = g

    for neighbor, cost in graph.neighbors(node):
        new_g = g + cost
        if neighbor in closed and closed[neighbor] <= new_g:
            continue

        heapq.heappush(open_list, (new_g + h(neighbor), neighbor, new_g, (node,
parent)))

    return None

# Heuristic function (h)
def heuristic(node):
    h_values = {
        'A': 6,
        'B': 5,
        'C': 3,
        'D': 3,
        'E': 1,
        'F': 0
    }
    return h_values.get(node, math.inf)

# Graph initialization
graph = Graph()
edges = [
    ('A', 'B', 1),
    ('A', 'C', 4),
    ('B', 'D', 2),
    ('B', 'E', 5),
    ('C', 'F', 3),
    ('D', 'E', 1),
    ('E', 'F', 1)
]
for u, v, cost in edges:
    graph.add_edge(u, v, cost)

# Start and goal nodes
start_node = 'A'
goal_node = 'F'

```

```
# RBFS solution
path_rbfs, _ = rbfs(graph, start_node, goal_node, math.inf, heuristic)
print("RBFS Path:", path_rbfs)

# SMA* solution
path_sma = sma_star(graph, start_node, goal_node, heuristic)
print("SMA* Path:", path_sma)
```

Muhammed Emin Korkunc - All Rights Reserved